

TP1 et 2

Gestion de chaîne de caractères –

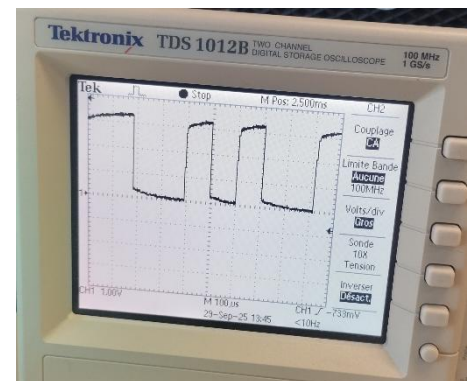
Norme NMEA

Les liaisons séries

- Au repos, la ligne est en tension à 3,3V, soit en position haute, ce qui correspond à un bit de valeur 1. En effet, si la ligne au repos valait 0, le module ne pourrait pas savoir si la ligne est toujours active ou non.
- D'après la partie 1 de l'énoncé du TP, le module utilisé ici émet les trames avec un débit D de 9600 bits par secondes. La durée T d'un seul bit est donc :

$$T = \frac{1}{D} = \frac{1}{9600} = 104,2 \mu s$$

- L'état de repos de la trame étant un bit 1, le passage à l'état 0 permet de « prévenir » de l'arrivée de la trame. En effet, sans cette synchronisation, si le premier bit de la trame était 1, il se confondrait avec l'état de repos de la ligne et donc la trame ne serait pas correctement déchiffrée.



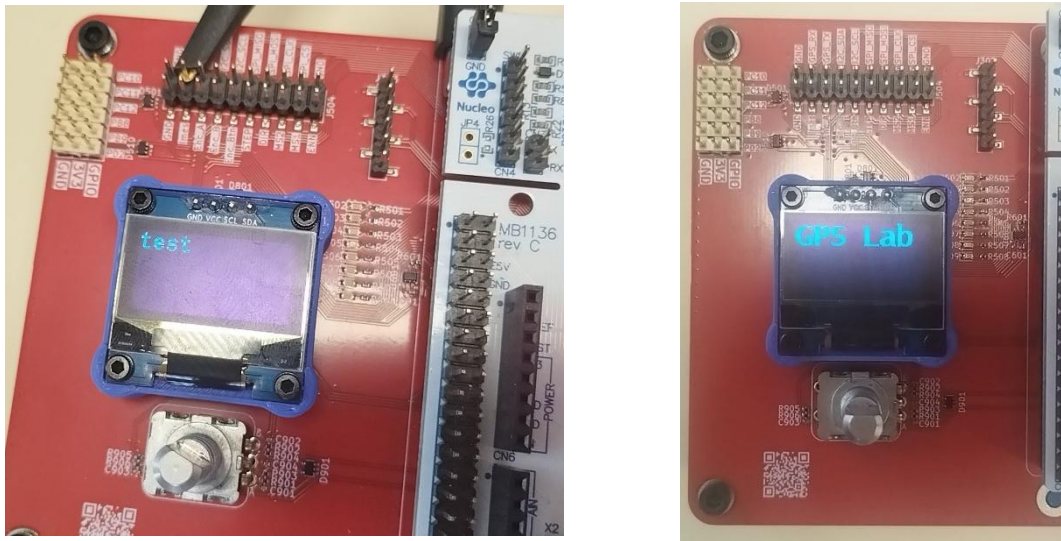
Extrait de la trame aperçue à l'oscilloscope

- Sachant qu'il y a un bit de start et un bit de stop en plus des 8 bits qui composent un octet, il faut alors envoyer 10 bits pour émettre un octet au module.
- Puisqu'il faut envoyer 10 bits pour émettre un octet, on en déduit que le débit binaire réel vaut :

$$D_{utile} = \frac{9600}{10} = 960 \text{ o/s}$$

Récupération des trames NMEA sur la liaison USART1

1. Vérification du paramétrage de la liaison série USART1



Images de l'écran OLED testé et fonctionnel

2. Boucle de recopie entrée / sortie

Le code proposé permet d'afficher le texte renseigné par l'utilisateur au fur et à mesure qu'il écrit

3. Fonction fillBuffer

Voici la traduction du pseudo-code en C

```
int fillBuffer(char* buffer){

    char received_char='\0';
    int total=0;

    for(int index=0; index<BUFFER_SIZE; index++){
        buffer[index]=0;
    }

    HAL_UART_Abort(&huart1);

    while(received_char!='$'){
        HAL_UART_Receive(&huart1, (uint8_t*)&received_char, 1, HAL_MAX_DELAY);
    }

    while(received_char!='\n' && total<BUFFER_SIZE-1){
        *buffer=received_char;
        HAL_UART_Receive(&huart1, (uint8_t*)&received_char, 1, HAL_MAX_DELAY);
        total++;
        buffer++;
    }
    *buffer=received_char;

    return total;
}
```

La variable `buffer` n'a pas besoin d'être remise à sa valeur initiale car, bien que les éléments du tableau soient modifiés en place grâce au principe de pointeur, `buffer` est quant à lui une copie du pointeur original puisqu'il est passé comme argument de la fonction.

Vérification de l'intégrité des trames

Code des fonctions `isGPGGA`, `extractChecksum`, et `calculateChecksum`:

```
int isGPGGA(char* frame){
    char gpgga[6]="$GPGGA";
    for(int i=0; i<6; i++){
        if(frame[i]!=gpgga[i]){
            return -1;
        }
    }
    //plutôt que de taper à la main une boucle for,
    //on pourrait utiliser la fonction strncmp du module string.h
    return 1;
}

int extractChecksum(char* buffer){
    while(*buffer!='*'){
        buffer++;
    }
    return (buffer[1]&0x0f)*16 + (buffer[2]&0x0f);
}

int calculateChecksum(char* buffer){
    int checksum=0;
    buffer++;
    while(*buffer!='*'){
        checksum=checksum^(*buffer);
        buffer++;
    }
    return checksum;
}
```

Dans la ligne 6 du pseudo-code, le calcul prend les deux caractères suivant celui pointé par `buffer`, qui à ce moment du code pointe le caractère '*' désignant le séparateur de checksum dans une trame NMEA ; `buffer[1]` et `buffer[2]` permettent donc d'extraire la checksum de la trame, stockée par deux caractères décrivant un nombre hexadécimal, puis le reste du calcul permet de le convertir en véritable entier hexadécimal.

Grâce aux trois fonctions précédentes, on peut coder la fonction **checkFrame** :

```
int checkFrame(char* buffer){
    if(isGPGGA(buffer)==1 &&
    calculateChecksum(buffer)==extractChecksum(buffer)){
        return 1;
    }
    else{
        return 0;
    }
}
```

On adapte ensuite le code de test pour utiliser cette nouvelle fonction :

```
void loop(){
    int size = fillBuffer(buffer);
    printf("Just received %d bytes : %s", size, buffer);
    if(checkFrame(buffer)==1){
        printf("Trame GPGGA\r\n");
    }
    else{
        printf("Trame invalide\r\n");
    }
    printf("\r\n");
}
```

Et en exécutant le code, la console nous informe bien que seule une trame sur trois est correcte, correspondant à ce qui est attendu

```
Just received 67 bytes : $GPGGA,084440.477,4150.109,N,08737.751,W,1,12,1.0,0.0,M,0.0,M,,*74
Trame GPGGA

Just received 62 bytes : $GPGSA,A,3,01,02,03,04,05,06,07,08,09,10,11,12,1.0,1.0,1.0*30
Trame invalide

Just received 63 bytes : $GPRMC,084440.477,A,4150.109,N,08737.751,W,,170723,000.0,W*60
Trame invalide
```

Résultat du test de la fonction **checkFrame**

Récupération de l'information au format flottant

Code de la fonction `getLatitude` :

```
int find_coma(char* frame, int size, int num){
    int i=0;
    while(i<size && num>0){
        i++;
        if(frame[i]==','){
            num--;
        }
    }
    return i;
}

float getLatitude(char* frame){

    int second_coma = find_coma(frame, BUFFER_SIZE, 2);

    char* latitude = frame + second_coma+1;
    float calcul_latitude = 0;

    calcul_latitude += 10*(latitude[0]&0xf);
    calcul_latitude += latitude[1]&0xf;
    float tmp=0;
    tmp+=10*(latitude[2]&0xf);
    tmp+=latitude[3]&0xf;
    tmp+=0.1*(latitude[5]&0xf);
    tmp+=0.01*(latitude[6]&0xf);
    tmp+=0.001*(latitude[7]&0xf);
    tmp/=60;
    calcul_latitude += tmp;

    calcul_latitude = (latitude[9]=='S') ? calcul_latitude*(-1) :
calcul_latitude;

    return calcul_latitude;
}
```

D'après la norme NMEA, la prochaine virgule de la trame devrait être située juste après le caractère 'N' ou 'S' changeant le signe du résultat, soit le 11ème caractère après la deuxième virgule trouvée précédemment.

La fonction `getLongitude` est similaire :

```
float getLongitude(char* frame){  
  
    int fourth_coma = find_coma(frame, BUFFER_SIZE, 4);  
  
    char* longitude = frame + fourth_coma+1;  
    float calcul_longitude = 0;  
  
    calcul_longitude += 100*(longitude[0]&0xf) + 10*(longitude[1]&0xf) +  
(longitude[2]&0xf);  
    float tmp=0;  
    tmp+=10*(longitude[3]&0xf);  
    tmp+=longitude[4]&0xf;  
    tmp+=0.1*(longitude[6]&0xf);  
    tmp+=0.01*(longitude[7]&0xf);  
    tmp+=0.001*(longitude[8]&0xf);  
    tmp/=60;  
    calcul_longitude += tmp;  
  
    calcul_longitude = (longitude[10]=='W') ? calcul_longitude*(-1) :  
calcul_longitude;  
  
    return calcul_longitude;  
}
```

On obtient des valeurs cohérentes lors des tests

```
Just received 67 bytes : $GPGBGA,084440.477,4150.109,N,08737.751,W,1,12,1.0,0.0,M,0.0,M,,*74  
Trame GPGBGA  
Latitude = 41.835152  
Longitude = -87.629181  
  
Just received 62 bytes : $GPGBSA,A,3,01,02,03,04,05,06,07,08,09,10,11,12,1.0,1.0,1.0*30  
Trame invalide  
  
Just received 63 bytes : $GPRMC,084440.477,A,4150.109,N,08737.751,W,,170723,000.0,W*60  
Trame invalide
```

Résultat du test de récupération
de la longitude et la latitude

Affichage de l'information et test du module réel

Malheureusement, le GPS externe n'a pas réussi à capter de signal à l'intérieur de la salle

```
Just received 43 bytes : $GNGSA,A,1,,,,,,,,,25.5,25.5,25.5,1*01
Trame invalide

Just received 19 bytes : $GPGSV,1,1,00,0*65
Trame invalide

Just received 26 bytes : $GNRMC,,V,,,,,,,,,N,V*37
Trame invalide

Just received 16 bytes : $GNZDA,,,,,*56
Trame invalide

Just received 31 bytes : $GNGGA,,,,,0,00,25.5,,,,,*64
Trame invalide

Just received 43 bytes : $GNGSA,A,1,,,,,,,,,25.5,25.5,25.5,1*01
Trame invalide
```

Résultat du test d'utilisation du GPS réel

Recherche de la ville la plus proche

1. Création de la base de donnée

On définit la structure suivant dans un fichier .h :

```
typedef struct ville{  
  
    float latitude;  
    float longitude;  
  
    char* nom;  
    //sera utile pour l'affichage  
    //de la ville la plus proche partie 10.3  
  
}VILLE;
```

Et on peut ensuite définir des variables correspondant aux données de l'énoncé :

```
#include "cityDataBase.h"  
  
#define TAILLE_BdD 4  
  
const VILLE kiev = {50.4501,30.5234,"Kiev"};  
const VILLE cergy = {49.033,2.066,"Cergy"};  
const VILLE chicago = {41.88,-87.623,"Chicago"};  
const VILLE barcelone = {41.385,2.173,"Barcelone"};  
  
VILLE base_de_donnée[TAILLE_BdD]={kiev, cergy, chicago, barcelone};
```

2. Calcul de distance

On traduit le calcul de norme en langage C :

```
#include <math.h> //nécessaire pour pouvoir appeler la fonction cos

float norme(VILLE* v, float latitude, float longitude){
    int norme = latitude - v->latitude;
    norme*=norme;
    float tmp = longitude - v->longitude;
    tmp*= cos((latitude - v->latitude) / 2);
    tmp*=tmp;
    norme+=tmp;
    return norme;
}
```

3. Recherche du minimum

Pour trouver l'élément du tableau le plus proche de la position donnée en argument, on peut créer un nouveau tableau qui va contenir les distances de chaque ville à la position, dans le même ordre que le tableau original (la distance à la première ville est en première position, etc...). Il ne reste ensuite plus qu'à faire une recherche de minimum sur ce tableau des distances.

```
int getMinIndex(float* array, int size){
    int index_min=0;
    for(int i=0; i<size; i++){
        if(array[i]<array[index_min]){
            index_min=i;
        }
    }
    return index_min;
}

VILLE* findNearest(VILLE tableau[], int size, float latitude, float longitude){
    float distance[size];
    for(int i=0; i<size; i++){
        distance[i] = norme(tableau+i, latitude, longitude);
    }
    return tableau+getMinIndex(distance, size);
}
```

On peut ensuite réaliser un test avec la base de donnée crée précédemment

```
void loop(){

    int size = fillBuffer(buffer);
    printf("Just received %d bytes : %s", size, buffer);

    if(checkFrame(buffer)==1){

        printf("Trame GPGBA\r\n");

        float latitude = getLatitude(buffer);
        float longitude = getLongitude(buffer);
        char* ville_la_plus_proche = (findNearest(base_de_donnée, TAILLE_BdD,
latitude, longitude))->nom;

        printf("Latitude = %f, Longitude = %f\r\n", latitude, longitude);
        printf("Ville la plus proche : %s\r\n", ville_la_plus_proche);

        snprintf(buffer,16,"Ville : %s",ville_la_plus_proche);
        ssd1315_Draw_String(0,30,buffer,&Font_7x10);
        ssd1315_Refresh();

    }

    else{
        printf("Trame invalide\r\n");
    }

    printf("\r\n");
}
```

Le test donne le résultat suivant dans la console et sur l'écran :

```
Just received 67 bytes : $GPGGA,084440.477,4150.109,N,08737.751,W,1,12,1.0,0.0,M,0.0,M,,*74
Trame GPGGA
Latitude = 41.835152, Longitude = -87.629181
Ville la plus proche : Chicago

Just received 62 bytes : $GPGSA,A,3,01,02,03,04,05,06,07,08,09,10,11,12,1.0,1.0,1.0*30
Trame invalide

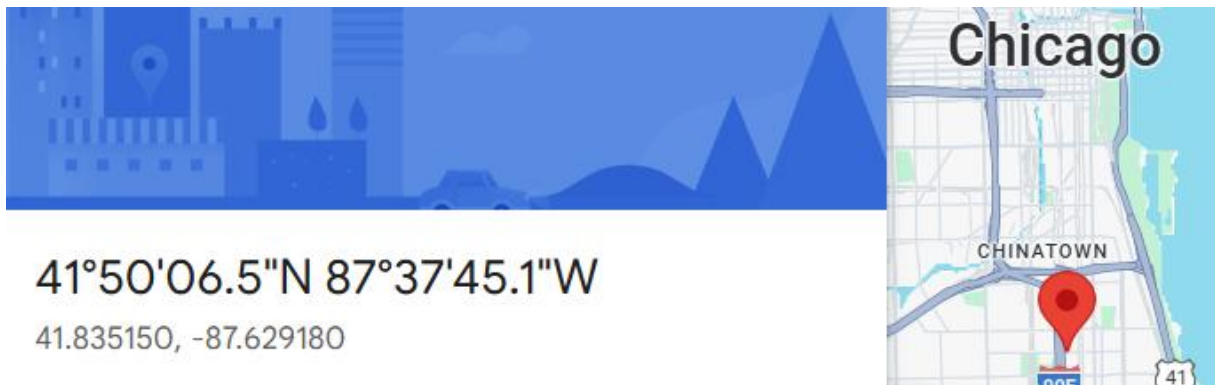
Just received 63 bytes : $GPRMC,084440.477,A,4150.109,N,08737.751,W,,170723,000.0,W*60
Trame invalide
```

Résultat du test de recherche de la ville la plus proche
(sur la console)



Résultat du test de
recherche de la ville la plus
proche (sur l'écran)

Cela est cohérent puisque la trame décrit un point placé dans la ville de Chicago



Emplacement décrit par la trame trouvé
sur une application de localisation